Министерство науки и высшего образования Российской Федерации
Национальный исследовательский университет ИТМО
Мегафакультет компьютерных технологий и управления
Факультет программной инженерии и компьютерной техники

Отчет по лабораторной работе № 2

по курсу «Механизмы надежности и безопасности вычислительных систем»

Тема: «Моделирование и верификация в среде SPIN»

Выполнил студент группы P4219 _____ Д. Л. Симоновский
(подпись)

Руководитель _____ В. Ю. Пинкевич
(подпись)

28 октября 2026 г.

Санкт-Петербург
2026

Содержание

1 Задача **2**

2 Ход работы **2**
2.1 Описание моделируемой системы 2
2.2 Взаимодействие процессов и принятые предположения 3
2.3 Реализация модели на языке Promela 5
2.4 Описание верифицируемых свойств 12
2.5 Автоматизация проверки 18
2.6 Верификация модели 21

3 Выводы **23**

Список использованных источников **24**

1. Задача

В рамках лабораторной работы требуется получить практические навыки моделирования взаимодействующих процессов на языке Promela и проверки корректности их работы с использованием верификатора SPIN. Для этого необходимо выбрать некоторую систему и выполнить верификацию алгоритма ее функционирования.

Для выбранной системы требуется составить текстовое описание, позволяющее понять состав участников, порядок их взаимодействия и правила обработки событий. При необходимости описание следует дополнить схемами взаимодействия процессов и алгоритмов их работы. На основе этого описания необходимо построить модель на языке Promela и пояснить принятые при моделировании предположения.

Далее требуется сформулировать не менее десяти свойств системы и представить их в виде LTL-формул. Если для проверки свойства недостаточно исходных переменных модели, необходимо предусмотреть дополнительные средства наблюдения за ее состоянием. Подготовленные свойства следует проверить в среде SPIN, после чего проанализировать результаты, описать выявленные ошибки или подтвердить соответствие модели сформулированным требованиям.

В заключении необходимо указать, какие предположения оказались существенными для верификации, потребовалось ли уточнять модель или свойства и в каких границах полученные результаты характеризуют работу выбранной системы.

2. Ход работы

2.1. Описание моделируемой системы

В качестве системы для верификации выбран обмен запросами и ответами по протоколу Modbus. В рамках работы рассматривается небольшая сеть, состоящая из одного ведущего устройства Master и трех ведомых устройств Slave с адресами 1, 2 и 3. Master формирует запрос, все Slave получают его копию, после чего каждый из них определяет, требуется ли обрабатывать этот запрос.

Основной операцией является запись одного регистра с кодом функции `0x06`. У каждого Slave предусмотрены два регистра с адресами 0 и 1. Если запрос содержит адрес конкретного устройства, запись выполняет только это устройство. После успешной записи оно возвращает ответ, повторяющий код функции, адрес регистра и записанное значение. Такой порядок обработки соответствует назначению функции Write Single Register [2].

Кроме адресной записи, модель предусматривает широковещательный запрос с адресом 0. Его обрабатывают все три Slave, однако ответ Master не отправляется. Для проверки отсутствующего получателя используется адрес 4: такого устройства в сети нет, поэтому запрос остается без ответа.

Перед выполнением записи Slave проверяет адрес получателя, корректность CRC, код функции и адрес регистра. При неверной CRC кадр игнорируется. Неподдерживаемая функция приводит к исключению с кодом 1, а обращение к несуществующему регистру — с кодом 2. Исключение возвращается только для адресного запроса; при широковещательной передаче устройство сохраняет молчание и в случае ошибки.

Таблица 2.1. Основные параметры модели

Параметр	Принятое значение
Состав участников	Один Master и три Slave
Адрес получателя	0 — всем; 1–3 — конкретному Slave; 4 — отсутствующему устройству
Адрес регистра	0 или 1; значение 2 используется как ошибочное
Значение для записи	1 или 2; начальные значения регистров равны 0
Код функции	6 — поддерживаемая запись; 3 — неподдерживаемая в этой модели функция
Корректность CRC	Логический признак `crc_ok`
Число активных запросов	Одна транзакция за раз

Master повторяет обмен в бесконечном цикле. Параметры очередного запроса выбираются недетерминированно, поэтому верификатор рассматривает различные запросы и допустимые порядки выполнения процессов. Сначала формируется корректная запись, после чего запрос остается без изменений либо в него добавляется одна ошибка: неверная CRC, ошибочный адрес регистра или неподдерживаемая функция. Одновременное сочетание нескольких таких ошибок генератором не задается.

2.2. Взаимодействие процессов и принятые предположения

На рисунке 2.1 приведен пример адресного обмена со Slave 2. Общий запрос доставляется каждому устройству через отдельный канал. Slave 1 и Slave 3 проверяют адрес и завершают обработку без изменения регистров. Slave 2 выполняет запись и помещает ответ в общий канал ответов. Уведомление о завершении отправляет каждый из трех обработчиков.

Каналы запросов имеют емкость в одно сообщение. Это позволяет каждому Slave независимо получить свою копию одного и того же кадра. Общий канал ответов также рассчитан на одно сообщение, поскольку адресным получателем может быть только одно устройство. Служебный канал завершения вмещает три уведомления. Эти уведомления нужны для согласования шагов модели и не являются сообщениями протокола Modbus.

Результат транзакции фиксируется после получения всех трех уведомлений. При адресном запросе наличие сообщения в канале ответов означает получение ответа, а его отсутствие — состояние тайм-аута. При широковещательном запросе Master сразу фиксирует завершение без ответа. Таким образом, тайм-аут описывает логический результат обмена; числовой интервал ожидания в модели не задается.

Передача кадра представлена доставкой полей сообщения, а CRC — готовым логическим признаком. Это позволяет сосредоточиться на выборе получателя, условиях записи, содержимом ответа и завершении транзакции. Значения регистров хранятся в переменных типа `byte`, а генератор выбирает только 1 и 2. Результат проверки относится к этим диапазонам и принятому порядку обмена.

Верифицируемые свойства должны показать, что корректные запросы выполняют нужную запись, ошибочные запросы сохраняют память, посторонние устройства и регистры не изменяются, ответы формируются правильно и начатый обмен обязательно достигает результата. Для этого далее рассматривается реализация модели и средства наблюдения за каждой транзакцией.

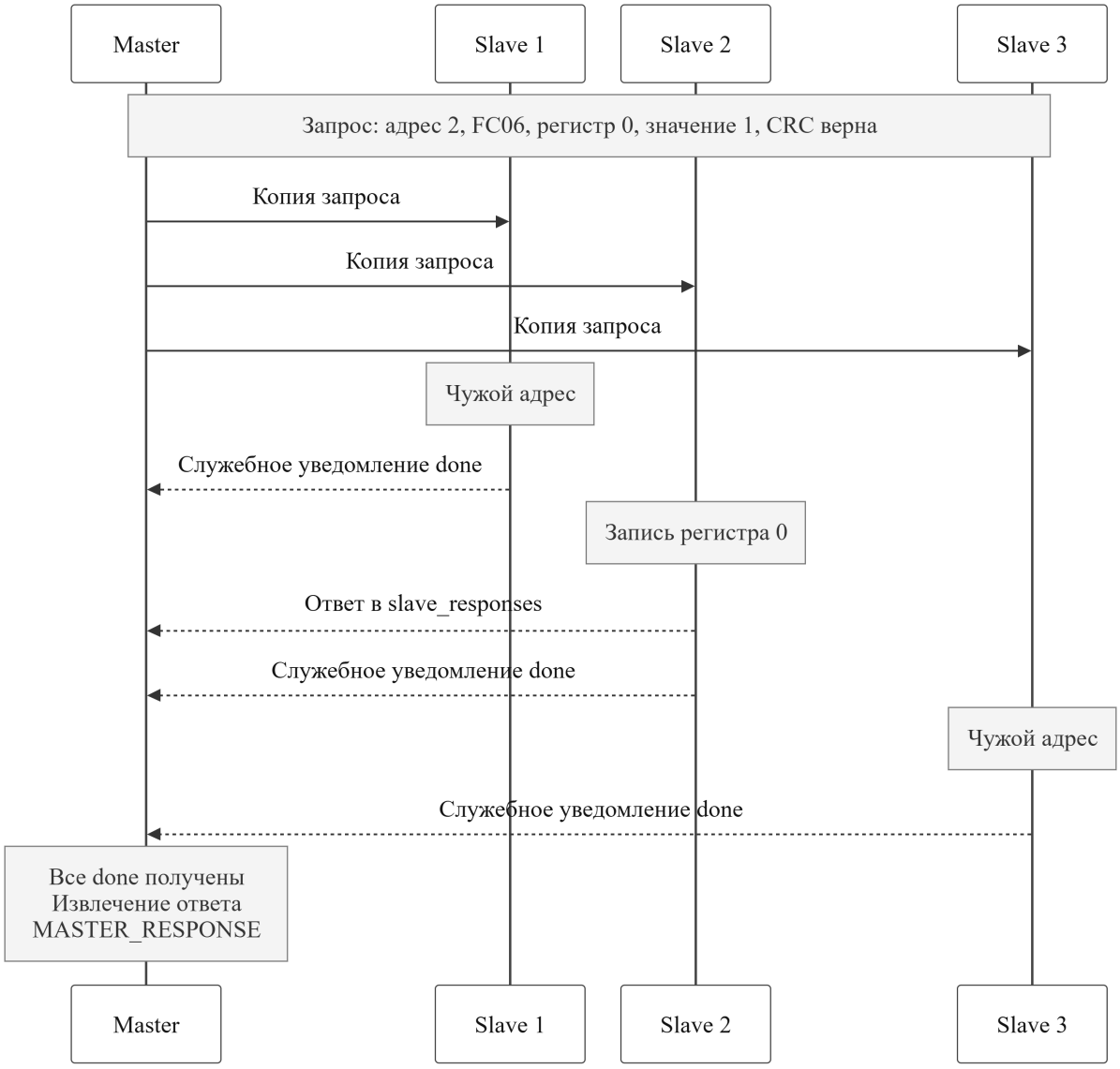


Рис. 2.1. Один из порядков взаимодействия процессов при запросе к Slave 2

2.3. Реализация модели на языке Promela

Модель системы находится в файле `modbus.pml`. В исходном файле приведены комментарии к переменным и основным этапам обмена; полный код доступен в репозитории проекта [1]. Далее рассматриваются функциональные части реализации. В листингах приведены фрагменты исходного кода без комментариев, а из обработчиков также убрана часть сообщений `printf`, предназначенных для просмотра обмена.

2.3.1. Состояние системы и запуск процессов

Для начала необходимо определить, какую информацию модель должна сохранять между шагами. У Master есть состояние текущего обмена и поля запроса. Память Slave представлена двумя массивами: индекс соответствует адресу устройства. Нулевой элемент массивов не используется, поэтому обращение к памяти Slave 2 записывается непосредственно через индекс 2.

Листинг 2.1. Состояние Master и память устройств

```
#define FC06 6
#define FC03 3
mtype = {
    MASTER_PREPARING, MASTER_WAITING, MASTER_RESPONSE,
    MASTER_TIMEOUT, MASTER_BROADCAST_DONE
};
mtype master_state = MASTER_PREPARING;
byte slave_address;
byte function_code;
byte register_address;
byte register_value;
bool crc_ok;
byte holding_register_0[4];
byte holding_register_1[4];
bool slave_write_performed[4];
```

Все числовые поля объявлены как `byte`. При создании модели их начальное значение равно нулю. После завершения транзакции содержимое регистров сохраняется, поэтому следующие запросы работают с результатами предыдущих записей. Флаг `slave_write_performed` показывает именно факт выполнения операции: повторная запись уже хранящегося значения также должна считаться записью.

Взаимодействие процессов организовано через каналы. Сообщение запроса содержит адрес получателя, код функции, адрес регистра, значение и признак CRC. В ответе последнее поле заменено кодом исключения. Нулевой код исключения означает нормальный ответ. Процесс `init` создает три экземпляра одного обработчика `Slave` и один экземпляр `Master`.

Листинг 2.2. Каналы обмена и начальный запуск

```
chan slave_requests[3] = [1] of { byte, byte, byte, byte, bool };
chan slave_responses = [1] of { byte, byte, byte, byte, byte };
chan slave_done = [3] of { bool };
init
{
    atomic {
        run Slave(1);
        run Slave(2);
        run Slave(3);
        run Master();
    }
}
```

Отдельные каналы запросов позволяют смоделировать получение общего кадра всеми устройствами. Если бы все Slave читали один обычный канал Promela, сообщение получил бы только один из них. В выбранной реализации адрес проверяет сам обработчик, а не механизм доставки.

2.3.2. Подготовка очередной транзакции

Работа Master представляет собой бесконечное повторение одного обмена. Алгоритм приведен на рисунке 2.2. Переход в `MASTER_PREPARING` обозначает границу между транзакциями. Затем Master очищает ответ и наблюдательные признаки предыдущего обмена, сохраняет значения регистров и выбирает новый запрос.

Снимок памяти необходим для последующего сравнения. Проверка «не было записи» и проверка «значение не изменилось» имеют разный смысл: запись того же значения не меняет память, но операция при этом выполняется. Поэтому используются и предыдущие значения, и отдельные флаги.

Листинг 2.3. Сброс признаков обмена и сохранение памяти

```
response_slave_address = 0;
response_function_code = 0;
response_register_address = 0;
response_register_value = 0;
response_exception_code = 0;
slave_write_performed[1] = false;
slave_write_performed[2] = false;
slave_write_performed[3] = false;
slave_processed[1] = false;
slave_processed[2] = false;
slave_processed[3] = false;
responses_sent = 0;
previous_register_0[1] = holding_register_0[1];
previous_register_0[2] = holding_register_0[2];
previous_register_0[3] = holding_register_0[3];
previous_register_1[1] = holding_register_1[1];
previous_register_1[2] = holding_register_1[2];
previous_register_1[3] = holding_register_1[3];
```

Далее Master выбирает параметры запроса. В конструкции `if` все ветви с присваиваниями допустимы, поэтому выбор является недетерминированным. Это задает множество вариантов поведения, которое исследует SPIN, а не конкретную случайную последовательность запросов.

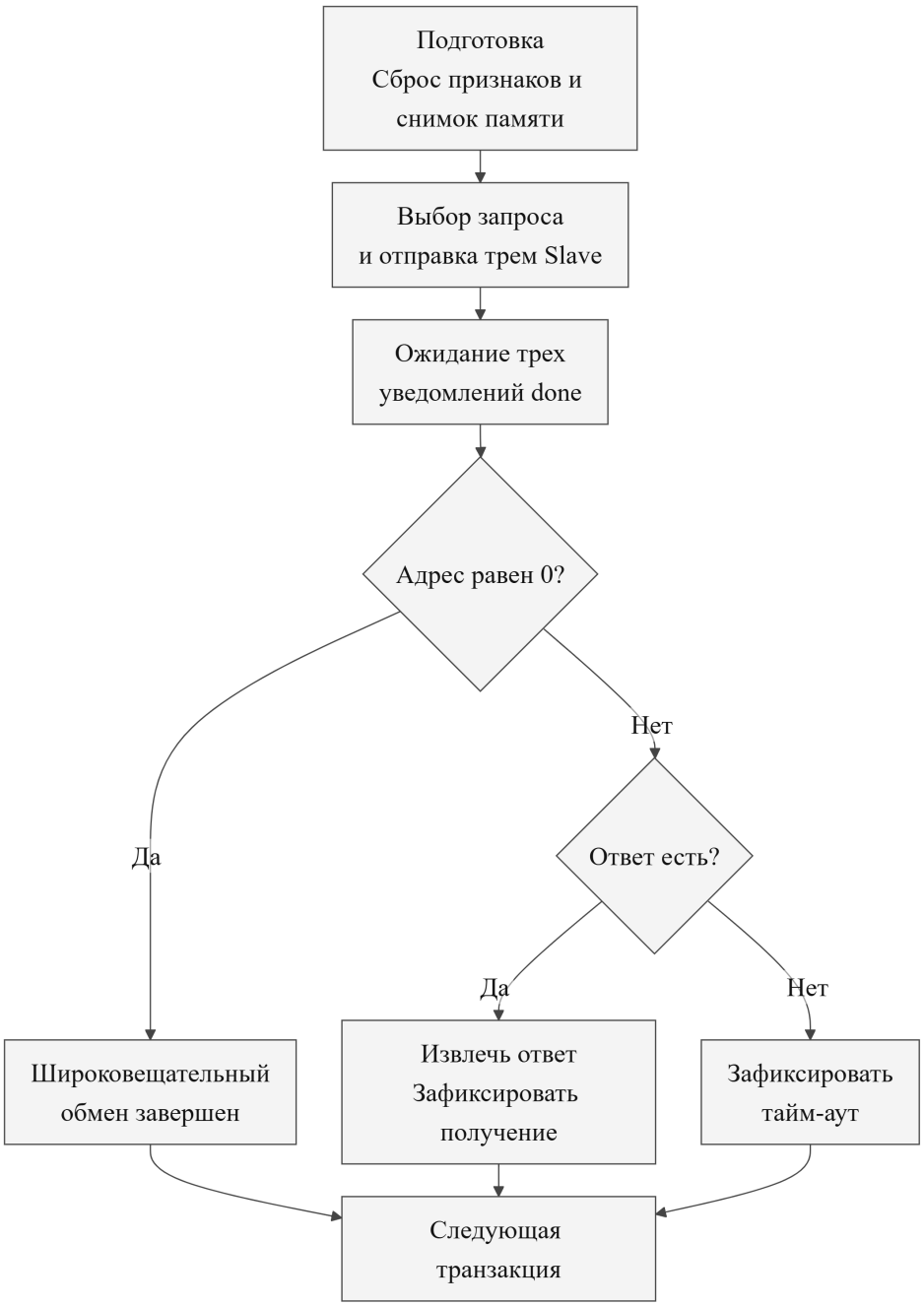


Рис. 2.2. Алгоритм работы процесса Master

Листинг 2.4. Выбор запроса и внесение одной ошибки

```
if
:: slave_address = 0
:: slave_address = 1
:: slave_address = 2
:: slave_address = 3
:: slave_address = 4
fi;
if
:: register_address = 0
:: register_address = 1
fi;
if
:: register_value = 1
:: register_value = 2
fi;
function_code = FC06;
crc_ok = true;
if
:: skip
:: crc_ok = false
:: register_address = 2
:: function_code = FC03
fi;
```

Код `FC03` соответствует функции чтения регистров в Modbus, но в рассматриваемой реализации эта функция специально не поддерживается. Здесь она служит примером запроса, на который требуется вернуть исключение. Проверка тем самым относится к устройству, реализующему только `FC06`.

После подготовки Master переходит в `MASTER_WAITING` и помещает одинаковые поля в три канала. Сброс признаков, выбор параметров и отправка объединены одним блоком `atomic`. Он сохраняет исключительное выполнение последовательности, пока ее операторы не блокируются [3]. К началу очередного обмена каналы запросов уже освобождены обработчиками.

Листинг 2.5. Передача копий одного запроса

```
master_state = MASTER_WAITING;
slave_requests[0]!slave_address, function_code,
    register_address, register_value, crc_ok;
slave_requests[1]!slave_address, function_code,
    register_address, register_value, crc_ok;
slave_requests[2]!slave_address, function_code,
    register_address, register_value, crc_ok;
```

2.3.3. Проверки на стороне Slave и выполнение записи

Каждый Slave работает в собственном бесконечном цикле и читает только свой канал `slave_requests[slave_id - 1]`. Поля принятого кадра сохраняются в локальных переменных. Поэтому один и тот же тип процесса можно использовать для всех трех устройств, передавая ему собственный адрес в параметре `slave_id`.

Алгоритм обработки показан на рисунке 2.3. Сначала проверяется, адресован ли запрос этому устройству или всем сразу. Затем проверяется CRC. Только после этого определяется допустимость функции и адреса регистра. Такой порядок позволяет

игнорировать поврежденный кадр до выполнения операции и формирования ответа.

В начале обработки `exception_code` равен нулю. При неподдерживаемой функции он получает значение 1, при ошибочном адресе регистра — 2. Если обе проверки пройдены, Slave записывает выбранный регистр и устанавливает `slave_write_performed[slave_id]`. Эти действия объединены в `atomic`, поэтому состояние памяти и признак записи наблюдаются согласованно.

Листинг 2.6. Проверка запроса, запись и формирование ответа

```
if
:: (request_slave_address == slave_id || request_slave_address == 0) ->
    if
    :: request_crc_ok ->
        if
        :: (request_function_code != FC06) -> exception_code = 1
        :: else ->
            if
            :: (request_register_address > 1) -> exception_code = 2
            :: else ->
                atomic {
                    if
                    :: (request_register_address == 0) ->
                        holding_register_0[slave_id] = request_register_value
                    :: else ->
                        holding_register_1[slave_id] = request_register_value
                    fi;
                    slave_write_performed[slave_id] = true;
                }
            fi
        fi;
        if
        :: (request_slave_address == 0) -> skip
        :: else ->
            if
            :: (exception_code == 0) ->
                atomic {
                    slave_responses!slave_id, request_function_code,
                        request_register_address, request_register_value, 0;
                    responses_sent++;
                }
            :: else ->
                atomic {
                    slave_responses!slave_id, (request_function_code + 128),
                        0, 0, exception_code;
                    responses_sent++;
                }
            fi
        fi
    :: else ->
        printf("Slave %d: ignored (bad CRC)\n", slave_id)
    fi
:: else -> skip
fi;
```

Вторая часть этого фрагмента определяет ответ. Для широковещательного запроса выполняется `skip`: память может измениться, но сообщения в канале ответов не возникает. При адресной успешной записи возвращаются исходные поля запроса и

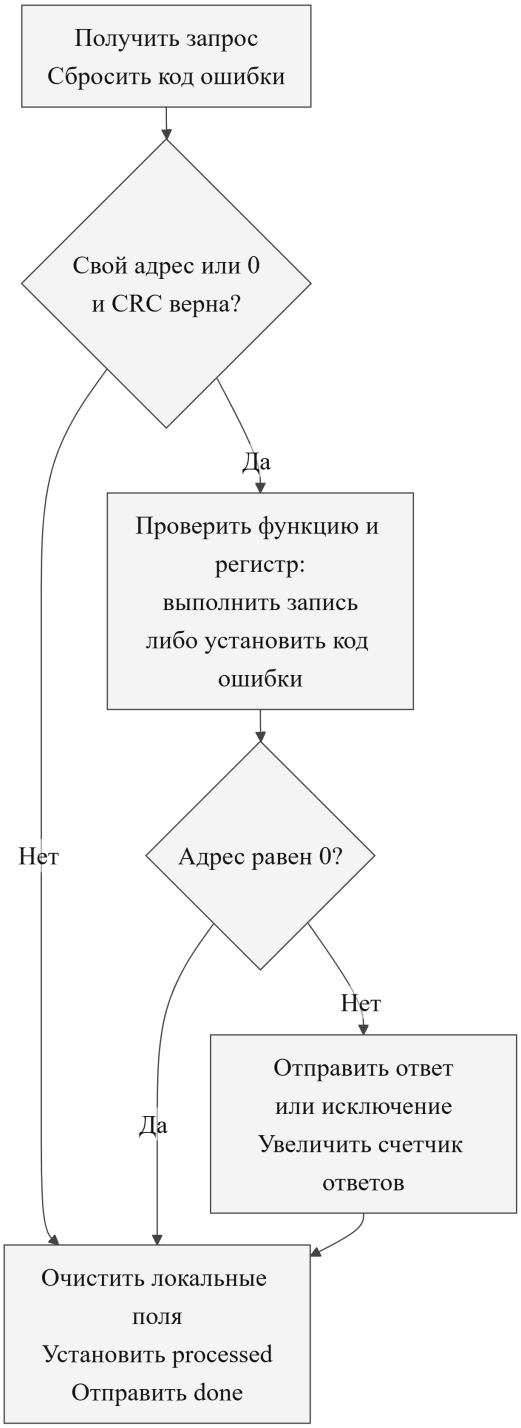


Рис. 2.3. Алгоритм обработки запроса одним Slave

нулевой код исключения.

Если обнаружена ошибка функции или регистра, к коду функции прибавляется 128, то есть устанавливается старший бит. В канале передаются также нулевые значения полей регистра и данных и код исключения. Эти нули относятся к фиксированной структуре сообщения модели: в ней нормальный ответ и исключение используют один тип канала.

Отправка ответа и увеличение `responses_sent` также выполняются совместно. Счетчик сохраняет факт отправки даже после того, как Master извлек сообщение из канала. Поэтому свойство «не более одного ответа» можно проверять на протяжении всей транзакции.

2.3.4. Согласование завершения обмена

После обработки запроса Slave устанавливает признак `slave_processed[slave_id]`. Далее очищаются локальные поля и отправляется служебное уведомление. Очистка локальных переменных не меняет память устройства, но устраняет из состояния завершившего обработку процесса данные старого кадра.

Листинг 2.7. Завершение обработки одного Slave

```
atomic {
    slave_processed[slave_id] = true;
    request_slave_address = 0;
    request_function_code = 0;
    request_register_address = 0;
    request_register_value = 0;
    request_crc_ok = false;
    exception_code = 0;
    slave_done!true;
}
```

Master читает три уведомления, после чего выбирает итоговое состояние. При адресе 0 это `MASTER_BROADCAST_DONE`. В остальных случаях проверяется наличие ответа: он извлекается в глобальные поля и фиксируется `MASTER_RESPONSE`, либо выбирается `MASTER_TIMEOUT`.

Листинг 2.8. Получение результата транзакции

```
slave_done?true;
slave_done?true;
slave_done?true;
if
:: (slave_address == 0) ->
    master_state = MASTER_BROADCAST_DONE;
:: else ->
    if
    :: (len(slave_responses) > 0) ->
        slave_responses?response_slave_address, response_function_code,
            response_register_address, response_register_value,
            response_exception_code;
        master_state = MASTER_RESPONSE;
    :: else ->
        master_state = MASTER_TIMEOUT;
    fi
fi;
```

Ожидание трех уведомлений существенно для проверки свойств. Оно позволяет отличить отсутствие ответа от ситуации, когда нужный Slave еще не получил возможность выполнить свой шаг. Кроме того, при переходе к следующей транзакции все обработчики уже закончили предыдущую.

Для наблюдения за этими условиями дополнительно объявлены снимки памяти, признаки завершения и счетчик ответов. Они используются в свойствах системы, которые далее вынесены в отдельный файл.

Листинг 2.9. Данные для наблюдения за транзакцией

```
byte previous_register_0[4];
byte previous_register_1[4];
bool slave_processed[4];
byte responses_sent;
```

2.4. Описание верифицируемых свойств

Свойства вынесены в файл `properties.pml`, который подключает модель командой `#include "modbus.pml"`. Всего сформулировано 14 LTL-свойств. Они проверяют обработку корректных и ошибочных запросов, сохранность памяти, ответы устройств и завершение обмена.

В формулах используются оператор `[]` со смыслом «всегда», импликация `->` и логические связки. Для свойства прогресса дополнительно используется сильный оператор `U`. Формулы записаны как требования к поведению модели. При генерации верификатора SPIN автоматически строит по отрицанию выбранной формулы автомат `never claim`, который ищет нарушающее ее выполнение [4]. Отдельные `assert`-выражения и написанный вручную `never claim` в проекте не используются.

Важной особенностью является выбор момента проверки. Условия, описывающие готовый результат, проверяются при `finished`. Ограничения, которые должны соблюдаться и во время обработки, используют `active`. Это позволяет не сравнивать поля разных транзакций при подготовке нового запроса. При этом выполнение условий после завершения само по себе еще не гарантирует, что транзакция завершится: этому посвящено отдельное свойство прогресса.

2.4.1. Игнорирование запроса с ошибочной CRC

Первое свойство требует, чтобы кадр с неверной CRC не приводил ни к записи, ни к ответу. Для проверки вводится макрос `finished`, обозначающий любое из трех итоговых состояний Master. Макрос `no_writes` проверяет отсутствие операций записи у всех устройств. `unchanged(s)` сравнивает оба регистра одного Slave со снимком, а `all_unchanged` распространяет сравнение на все три устройства.

Листинг 2.10. Признаки завершения и сохранности памяти

```
#define finished (master_state == MASTER_RESPONSE || master_state == MASTER_TIMEOUT ||
    master_state == MASTER_BROADCAST_DONE)
#define no_writes (!slave_write_performed[1] && !slave_write_performed[2] &&
    !slave_write_performed[3])
#define unchanged(s) (holding_register_0[s] == previous_register_0[s] && \
    holding_register_1[s] == previous_register_1[s])
#define all_unchanged (unchanged(1) && unchanged(2) && unchanged(3))
```

Листинг 2.11. Свойство 1 Обработка ошибочной CRC

```
ltl p01_crc_ignored {
    [] ((finished && !crc_ok) ->
        (responses_sent == 0 && no_writes && all_unchanged &&
            ((slave_address == 0 && master_state == MASTER_BROADCAST_DONE) ||
             (slave_address != 0 && master_state == MASTER_TIMEOUT))))
}
```

Формула проверяется во всех состояниях выполнения. Если обмен завершен и CRC неверна, число ответов должно быть равно нулю, флаги записи должны оставаться сброшенными, а память — совпадать со снимком. Для адреса 0 ожидается завершение широковещательного обмена, для любого другого адреса — тайм-аут.

2.4.2. Обращение к отсутствующему устройству

Второе свойство относится к адресу 4, для которого процесс Slave не создан. Такой запрос должен завершаться тайм-аутом, не затрагивая память существующих устройств и не создавая ответов.

Листинг 2.12. Свойство 2 Отсутствующий получатель

```
ltl p02_absent_slave {
    [] ((finished && slave_address == 4) ->
        (master_state == MASTER_TIMEOUT && responses_sent == 0 && no_writes &&
    all_unchanged))
}
```

Проверка не ограничивается корректными кадрами: при любом запросе к отсутствующему адресу результат должен удовлетворять указанным условиям. Это позволяет отдельно проверить фильтрацию получателя всеми Slave.

2.4.3. Отсутствие ответа на широковещательный запрос

Широковещательный запрос не должен создавать ответ даже во время обработки. Для этого используется `active`: он истинен в состоянии ожидания и в любом итоговом состоянии транзакции.

Листинг 2.13. Признак активной транзакции

```
#define active (master_state == MASTER_WAITING || finished)
```

Листинг 2.14. Свойство 3 Молчание при широковещательной передаче

```
ltl p03_broadcast_silent {
    [] ((active && slave_address == 0) ->
        (responses_sent == 0 && len(slave_responses) == 0))
}
```

Условие проверяет одновременно счетчик отправленных ответов и длину канала. Поэтому нарушение нельзя скрыть последующим извлечением сообщения: после отправки значение счетчика уже не было бы нулевым.

2.4.4. Запись корректного широковещательного запроса

Для проверки записи вводится `valid_write`. Он требует верной CRC, функции `FC06` и существующего регистра, но не проверяет адрес получателя. Макрос `written(s)` устанавливает, что нужный регистр содержит переданное значение, а второй регистр этого Slave не изменился.

Листинг 2.15. Корректная запись и ее результат

```
#define valid_write (crc_ok && function_code == FC06 && register_address <= 1)
#define written(s) ( \
    (register_address == 0 && holding_register_0[s] == register_value && \
        holding_register_1[s] == previous_register_1[s]) || \
    (register_address == 1 && holding_register_1[s] == register_value && \
        holding_register_0[s] == previous_register_0[s]))
```

Листинг 2.16. Свойство 4 Запись всеми Slave

```
ltl p04_broadcast_written {
    [] ((finished && slave_address == 0 && valid_write) ->
        (master_state == MASTER_BROADCAST_DONE &&
            slave_write_performed[1] && slave_write_performed[2] &&
    slave_write_performed[3] &&
            written(1) && written(2) && written(3)))
}
```

После завершения корректного запроса с адресом 0 все три устройства должны иметь установленный флаг записи и ожидаемое содержимое памяти. Отдельная проверка флагов существенна, если переданное значение уже находилось в регистре до запроса. Итоговое состояние Master должно соответствовать широковещательному завершению.

2.4.5. Выполнение корректной адресной записи

Макрос `addressed` обозначает адрес существующего устройства, то есть диапазон от 1 до 3. Пятое свойство требует, чтобы корректная запись такому устройству завершалась нормальным ответом и действительно выполнялась у выбранного Slave.

Листинг 2.17. Признак существующего адресного получателя

```
#define addressed (slave_address >= 1 && slave_address <= 3)
```

Листинг 2.18. Свойство 5 Успешная адресная запись

```
ltl p05_addressed_write {
    [] ((finished && addressed && valid_write) ->
        (master_state == MASTER_RESPONSE && response_exception_code == 0 &&
            ((slave_address == 1 && slave_write_performed[1] && written(1)) ||
             (slave_address == 2 && slave_write_performed[2] && written(2)) ||
             (slave_address == 3 && slave_write_performed[3] && written(3)))))
}
```

Дизъюнкция из трех ветвей связывает адрес запроса с нужным индексом в массивах. Для выбранного устройства проверяются факт записи и `written(s)`. Поля нормального ответа проверяются далее отдельной формулой, а сохранность остальных Slave — свойством изоляции.

2.4.6. Изоляция записи

Адресная операция должна изменять только выбранный регистр выбранного устройства. Макрос `isolated(s)` рассматривает два случая. Если Slave не является получателем, у него не должно быть ни операции записи, ни изменения памяти. Если запрос адресован ему, неизменным должен оставаться другой регистр, а при ошибочном адресе регистра — оба.

Листинг 2.19. Проверка изоляции одного Slave

```
#define isolated(s) ( \
    (slave_address == s || (!slave_write_performed[s] && unchanged(s))) && \
    (slave_address != s || \
        (register_address == 0 && holding_register_1[s] == previous_register_1[s]) || \
        (register_address == 1 && holding_register_0[s] == previous_register_0[s]) || \
        (register_address > 1 && unchanged(s))))
```

Листинг 2.20. Свойство 6 Изоляция адресной записи

```
ltl p06_write_isolation {
    [] ((finished && addressed) -> (isolated(1) && isolated(2) && isolated(3)))
}
```

Формула применяется ко всем завершенным адресным запросам, включая ошибочные. Совместная проверка трех экземпляров `isolated` охватывает и посторонние устройства, и невыбранную часть памяти получателя.

2.4.7. Содержимое нормального ответа

Седьмое свойство начинается с факта получения нормального ответа. В таком случае исходный запрос должен быть корректной адресной записью, а количество отправленных ответов — равняться одному.

Листинг 2.21. Свойство 7 Поля нормального ответа

```
ltl p07_normal_response {
    [] ((master_state == MASTER_RESPONSE && response_exception_code == 0) ->
        (addressed && valid_write && responses_sent == 1 &&
            response_slave_address == slave_address && response_function_code == FC06 &&
            response_register_address == register_address &&
            response_register_value == register_value))
}
```

Проверяется совпадение адреса Slave, адреса регистра и значения с запросом, а также код функции `FC06`. Таким образом, формула исключает нормальный ответ на поврежденный, широковещательный или неподдерживаемый запрос и проверяет содержимое уже принятого сообщения.

2.4.8. Исключение для неподдерживаемой функции

Для описания исключений используется `exception_response(code)`. Он проверяет состояние получения ответа, единственность отправки, совпадение адреса и установку старшего бита кода функции. Дополнительно проверяются нулевые поля регистра и данных и требуемый код исключения.

Листинг 2.22. Общий вид ответа с исключением

```
#define exception_response(code) (master_state == MASTER_RESPONSE && \
    responses_sent == 1 && response_slave_address == slave_address && \
    response_function_code == function_code + 128 && \
    response_register_address == 0 && response_register_value == 0 && \
    response_exception_code == code)
```

Листинг 2.23. Свойство 8 Неподдерживаемая функция

```
ltl p08_illegal_function {
    [] ((finished && addressed && crc_ok && function_code != FC06) ->
        (exception_response(1) && no_writes && all_unchanged))
}
```

Если адресный запрос имеет верную CRC, но его функция отличается от `FC06`, после завершения требуется исключение 1. Операций записи быть не должно, все регистры должны совпадать со снимком. В генерируемых запросах неподдерживаемая функция представлена кодом `FC03`.

2.4.9. Исключение для несуществующего регистра

Девятое свойство использует тот же макрос ответа, но требует код исключения 2. Предпосылка включает верную CRC и поддерживаемую функцию, поэтому проверяется именно ошибка адреса регистра.

Листинг 2.24. Свойство 9 Ошибочный адрес регистра

```
ltl p09_illegal_register {
    [] ((finished && addressed && crc_ok && function_code == FC06 &&
            register_address > 1) ->
        (exception_response(2) && no_writes && all_unchanged))
}
```

После такого запроса ответ должен иметь правильное оформление исключения, а память всех Slave должна остаться неизменной. Флаги записи позволяют проверить также отсутствие попытки записи без видимого изменения значения.

2.4.10. Допустимость каждой выполненной записи

Макрос `authorized(s)` связывает установленный флаг записи с корректностью запроса и его адресом. Если Slave выполнял запись, запрос должен удовлетворять `valid_write` и быть адресован этому устройству либо всем устройствам сразу.

Листинг 2.25. Условие допустимости записи

```
#define authorized(s) (!slave_write_performed[s] || (valid_write && (slave_address == 0
    || slave_address == s)))
```

Листинг 2.26. Свойство 10 Запись только по допустимому запросу

```
ltl p10_write_authorized {
    [] (active -> (authorized(1) && authorized(2) && authorized(3)))
}
```

В отличие от свойств готового результата, это условие проверяется во всех активных состояниях. Название `authorized` здесь означает допустимость операции по правилам модели: отдельный механизм аутентификации участников в нее не вводится.

2.4.11. Единственность ответа

Один запрос не должен приводить к нескольким ответам. Для проверки используется накопительный счетчик `responses_sent`, который сбрасывается только при подготовке следующей транзакции.

Листинг 2.27. Свойство 11 Не более одного ответа

```
ltl p11_single_response {
    [] (active -> (responses_sent <= 1))
}
```

Условие допускает ноль ответов, что необходимо для широковещательной передачи, неверной CRC и отсутствующего получателя. Одновременно оно запрещает повторную отправку в любой момент активной транзакции.

2.4.12. Обязательное завершение начатой транзакции

Проверок конечного результата недостаточно: они могли бы выполняться формально и при отсутствии завершенных обменов. Поэтому двенадцатое свойство требует прогресса из `MASTER_WAITING`.

Листинг 2.28. Свойство 12 Прогресс транзакции

```
ltl p12_transaction_progress {
    [] ((master_state == MASTER_WAITING) -> ((master_state == MASTER_WAITING) U
    finished))
}
```

Сильный оператор `U` требует, чтобы `finished` обязательно наступил, а до этого Master оставался в состоянии ожидания. Таким образом, запрос нельзя бесконечно обрабатывать или заменить новым запросом до фиксации результата.

Проверка выполняется без дополнительного параметра справедливости планирования `-f`. В данной модели каждый Slave проходит конечную обработку и затем ожидает следующий запрос, а Master ожидает уведомления. Поэтому бесконечная работа постороннего участника внутри одной транзакции отдельным циклом не предусмотрена. Полученный результат относится к этой структуре процессов.

2.4.13. Согласованность состояния завершения

Следующее свойство уточняет смысл `finished`. К моменту фиксации результата все обработчики должны завершить запрос, а сообщения текущей транзакции должны быть извлечены из каналов.

Листинг 2.29. Свойство 13 Завершение всех обработчиков и очистка каналов

```
ltl p13_completion_clean {
    [] (finished ->
        (slave_processed[1] && slave_processed[2] && slave_processed[3] &&
            len(slave_requests[0]) == 0 && len(slave_requests[1]) == 0 &&
            len(slave_requests[2]) == 0 && len(slave_done) == 0 &&
            len(slave_responses) == 0))
}
```

Проверяются три признака `slave_processed`, три очереди запросов, канал служебных уведомлений и канал ответов. Это условие не требует обнуления регистров: полезные данные сохраняются между транзакциями, а очищается только обмен сообщениями.

2.4.14. Ошибочный широковещательный запрос

Последнее свойство объединяет требования к широковещательной передаче и обработке ошибок. Если адрес равен нулю, но `valid_write` не выполняется, обмен должен завершиться без ответа и изменения памяти.

Листинг 2.30. Свойство 14 Ошибочная широковещательная передача

```
ltl p14_invalid_broadcast {
    [] ((finished && slave_address == 0 && !valid_write) ->
        (master_state == MASTER_BROADCAST_DONE && responses_sent == 0 &&
            len(slave_responses) == 0 && no_writes && all_unchanged))
}
```

Формула охватывает все предусмотренные генератором ошибки широковещательного запроса. Ни один Slave не должен выполнить запись, счетчик и канал ответов должны остаться пустыми, а Master должен зафиксировать `MASTER_BROADCAST_DONE`. Совместно с четвертым свойством это описывает оба исхода широковещательной обработки: запись корректного запроса и отсутствие изменений при ошибке.

2.5. Автоматизация проверки

Для запуска верификации используется скрипт `run.py` на языке Python. Он работает со стандартной библиотекой и объединяет подготовку файлов, вызов SPIN, компиляцию верификаторов и обработку результатов. Проверка запускается командой `python run.py` из корня проекта.

2.5.1. Подготовка файлов и запуск инструментов

Работа начинается в функции `main`. В каталоге `verification_results` создается отдельная папка `run_*`, чтобы артефакты нового запуска не перезаписывали предыдущие. Функция `prepare_files` копирует в него модель, свойства и конфигурацию, после чего читает пути к SPIN и GCC из `config.json`. Имена проверяемых свойств извлекаются из объявлений `ltl` регулярным выражением.

Листинг 2.31. Подготовка входных данных проверки

```
def prepare_files(folder: Path) -> tuple[Path, Path, list[str]]:
    for filename in ("modbus.pml", "properties.pml", "config.json"):
        shutil.copy2(ROOT / filename, folder / filename)
    config = json.loads((folder / "config.json").read_text(encoding="utf-8"))
    spin = Path(os.path.expandvars(config["spin"]))
    cc = Path(os.path.expandvars(config["cc"]))
    os.environ["PATH"] = str(cc.parent) + os.pathsep + os.environ.get("PATH", "")
    source = (folder / "properties.pml").read_text(encoding="utf-8")
    properties = re.findall(r"^ltl\s+(\w+)\s*\{", source, re.MULTILINE)
    if not properties:
        raise RuntimeError("В properties.pml не найдены LTL-свойства")
    return spin, cc, properties
```

Функция `run` выполняет внешнюю команду через `subprocess.run`, сохраняет ее полный вывод в отдельный лог и проверяет код завершения. Для каждой команды установлен лимит в 180 секунд. Ошибка запуска, ненулевой код или превышение времени не позволяют считать проверку успешной.

Далее `build_verifiers` сохраняет версии инструментов, вызывает `spin -a properties.pml` и дважды компилирует полученный `pan.c`. Один исполняемый файл предназначен для структурной проверки модели, второй — для проверки LTL-свойств. Основные команды приведены ниже.

Листинг 2.32. Генерация и запуск верификаторов

```
spin -a properties.pml
gcc -O2 -DCOLLAPSE -DSAFETY -DNOCLAIM -o pan_safety.exe pan.c
gcc -O2 -DCOLLAPSE -o pan_ltl.exe pan.c
pan_safety.exe -m1000000 -w22 -q -b
pan_ltl.exe -a -N p01_crc_ignored -m1000000 -w22 -b
```

Параметры `-DSAFETY -DNOCLAIM` отключают автомат свойства и поиск циклов в структурной проверке. В LTL-режиме `-a` включает поиск принимающих циклов, а `-N` выбирает формулу по имени. `-DCOLLAPSE` включает сжатие состояний; приближенный режим `BITSTATE` не используется [5].

Максимальная глубина обхода задана как 1 000 000 шагов. Параметр `-b` делает превышение глубины ошибкой, а `-q` в структурной проверке требует пустых каналов в допустимом конечном состоянии. `-w22` задает начальный размер хеш-таблицы поиска. Сохраняется стандартное сокращение частичных порядков SPIN. Формулы не используют оператор следующего состояния `X`.

2.5.2. Обработка результатов и контрпримеров

Сам по себе нулевой код завершения программы еще не является подтверждением свойства. Функция `check_search_output` читает число ошибок, проверяет отсутствие сообщений о нехватке ресурсов и убеждается, что верификатор действительно выполнял полный поиск в нужном режиме.

Листинг 2.33. Проверка достоверности результата поиска

```
def check_search_output(name: str, output: str) -> int:
    errors = re.search(r"\berrors:\s*(\d+)", output)
    limits = (
        "max search depth too small", "depth limit reached", "out of memory",
        "cannot allocate", "vectorsz is too small", "time limit reached",
    )
    if not errors or any(text in output.lower() for text in limits):
        raise RuntimeError("Нет полного результата или достигнут предел ресурсов")
    if "Full statespace search for:" not in output:
        raise RuntimeError("SPIN не подтвердил полный поиск состояний")
    if name == "structure":
        if not re.search(r"invalid end states\s+\+", output):
            raise RuntimeError("Проверка конечных состояний отключена")
    else:
        if not re.search(r"never claim\s+\+", output):
            raise RuntimeError("LTL-проверка отключена")
        if not re.search(r"acceptance\s+cycles\s+\+", output):
            raise RuntimeError("Поиск принимающих циклов отключён")
    return int(errors.group(1))
```

Для структурной проверки должен быть включен поиск недопустимых конечных состояний. Для LTL должны быть активны `never claim` и поиск принимающих циклов. Затем функция `verify` различает найденное нарушение, успешную проверку и техническую ошибку.

Листинг 2.34. Формирование статуса одной проверки

```
def verify(name: str, command: list[str], folder: Path, spin: Path) -> tuple[str, str]:
    try:
        output = run(command, folder, name + ".log")
        error_count = check_search_output(name, output)
        if error_count > 0:
            save_counterexample(name, folder, spin)
            return "FAIL", f"Найден контрпример: {name}_trace.log"
        if "search not completed" in output.lower():
            raise RuntimeError("Поиск не завершён")
        states = re.search(r"([\d.e+]+) states, stored", output)
        depth = re.search(r"depth reached (\d+)", output)
        if not states or not depth:
            raise RuntimeError("Не удалось прочитать статистику поиска")
        return "PASS", f"состояний {states.group(1)}, глубина {depth.group(1)}, ошибок 0"
    except (OSError, RuntimeError) as error:
        return "ERROR", f"{error}; см. {name}.log"
```

Если обнаружено нарушение, `save_counterexample` сохраняет `properties.pml.trail` под именем свойства и запускает SPIN с параметрами `-t -p -g -l -k`. В отдельный лог записывается пошаговое воспроизведение с состояниями процессов и переменных. Если поиск не завершился или возникла ошибка инструмента, результат получает статус `ERROR`, а не `PASS`.

Функция `run_checks` сначала проверяет структуру модели, затем последовательно запускает все найденные LTL-свойства. В конце сохраняется `report.txt` с результатами и общим количеством `PASS`, `FAIL` и `ERROR`. Код завершения скрипта равен 0 при полном успехе, 1 при нарушении свойства и 2 при технической ошибке. При наличии и нарушения, и технической ошибки возвращается 2.

2.6. Верификация модели

Проверка выполнена 28 октября 2026 года в Windows с использованием SPIN 6.5.1 и GCC 16.2.0. Для запуска использовался рассмотренный скрипт `run.py`. В отдельной папке запуска были сохранены копии исходных файлов, сгенерированный верификатор и логи всех этапов.

Сначала выполнялась структурная проверка без `never claim`, затем — отдельный запуск для каждой из 14 формул. Такой порядок позволяет разделить недопустимое конечное состояние самой модели и нарушение конкретного требования. При проверке использовались полный поиск, сжатие `COLLAPSE` и стандартное сокращение частичных порядков.

В отличие от запуска одного сценария, верификация исследует допустимые варианты запросов и взаимодействия процессов. Память не сбрасывается между транзакциями, поэтому проверяются также последующие обращения к уже измененным регистрам. Начальное состояние при этом фиксировано: все регистры равны нулю, Master находится в состоянии подготовки.

2.6.1. Результаты автоматического и отдельного запусков

Автоматический запуск выполнил 15 проверок и сформировал итог: `PASS=14, FAIL=0, ERROR=1`. Структурная проверка и 13 LTL-свойств завершились без нарушений. Для свойства `p12_transaction_progress` сработало ограничение Python-скрипта в 180 секунд: в этом запуске SPIN выполнил около 35 миллионов посещений состояний, но обход не был закончен. Поэтому результат `ERROR` отражал недостаточное время проверки, а не найденный контрпример.

Чтобы получить законченный результат для свойства прогресса, тот же собранный верификатор был запущен отдельно из папки проверки. Параметры поиска, модель и LTL-формула сохранены; ограничение времени, установленное в Python-обертке, к этой команде не применялось.

Листинг 2.35. Отдельная проверка свойства прогресса

```
pan_ltl.exe -a -N p12_transaction_progress -m1000000 -w22 -b
```

Отдельный поиск завершился за 214 секунд с результатом `errors: 0`. Было сохранено 28 849 450 состояний и выполнено около 43,1 миллиона их посещений. Таким образом, все 14 LTL-свойств получили законченный результат без нарушений; структурная проверка также пройдена. Контрпримеров не найдено. Итоги завершенных проверок приведены в таблице 2.2.

Таблица 2.2. Результаты завершенных проверок модели

Проверка	Состояний	Глубина	Время, с	Итог
`structure`	14 552 302	110 960	43,1	PASS
`p01_crc_ignored`	14 552 302	194 839	62,3	PASS
`p02_absent_slave`	14 552 302	194 839	69,3	PASS
`p03_broadcast_silent`	14 552 302	194 839	74,7	PASS
`p04_broadcast_written`	14 552 302	194 839	74,0	PASS
`p05_addressed_write`	14 552 302	194 839	64,7	PASS
`p06_write_isolation`	14 552 302	194 839	70,4	PASS
`p07_normal_response`	14 552 302	194 839	67,0	PASS
`p08_illegal_function`	14 552 302	194 839	72,4	PASS
`p09_illegal_register`	14 552 302	194 839	66,9	PASS
`p10_write_authorized`	14 552 302	194 839	69,2	PASS
`p11_single_response`	14 552 302	194 839	68,7	PASS
`p12_transaction_progress`*	28 849 450	194 839	214,0	PASS
`p13_completion_clean`	14 552 302	194 839	63,4	PASS
`p14_invalid_broadcast`	14 552 302	194 839	62,0	PASS

* Свойство p12 проверено отдельным запуском без ограничения времени Python-обертки.

В таблице указано число сохраненных состояний из поля `states, stored` и время, сообщаемое самим SPIN. При LTL-проверке исследуется поведение модели совместно с автоматом свойства, поэтому число состояний может различаться между формулами. Достигнутая глубина оказалась меньше заданного предела в 1 000 000 шагов.

Большинство LTL-проверок потребовало от 62 до 74,7 секунды и около 1 208 Мбайт памяти. Для свойства прогресса SPIN сообщил суммарное использование 2 358,511 Мбайт памяти. Общее время 15 успешных поисков составило примерно 19 минут, без учета генерации, компиляции и прерванной попытки проверки `p12`.

Полные логи находятся в папке запуска. Вместе с отчетом сохранены исходный итог скрипта `verification/automated_report.txt`, лог отдельной проверки `verification/p12_manual.log` и сводная статистика `verification/summary.json`. Сводка успешных поисков учитывает отдельную проверку `p12` и не заменяет исходный итог скрипта.

2.6.2. Существенные уточнения модели и свойств

Для проверки представленной версии изменять алгоритм Master и Slave или переписывать LTL-формулы не потребовалось. Потребовалось уточнить порядок выполнения верификации: свойство прогресса было проверено отдельно, поскольку фиксированного времени Python-скрипта не хватило для полного обхода.

При этом сама возможность содержательной проверки опирается на детали, уже предусмотренные в модели. Снимки памяти позволяют сравнивать состояния до и после запроса. Флаги записи отличают отсутствие операции от повторной записи того же значения. Счетчик ответов сохраняет сведения об отправке после извлечения сообщения из канала.

Три служебных уведомления задают момент, когда результат транзакции действительно можно оценивать. Без ожидания завершения обработчиков отсутствие ответа можно было бы принять за тайм-аут до того, как адресованный Slave выполнил свой

шаг. Состояние подготовки и условия `active` и `finished` также отделяют наблюдения текущего обмена от сброса признаков перед следующим запросом.

Следовательно, найденных противоречий между алгоритмом и сформулированными требованиями нет. Результат относится к выбранным диапазонам данных, начальному состоянию и последовательному выполнению транзакций. Отсутствие ошибок в этой модели подтверждает описанную логику обработки, но само по себе не является доказательством корректности полного устройства Modbus.

3. Выводы

В рамках лабораторной работы рассмотрена модель обмена Modbus с одним ведущим и тремя ведомыми устройствами. На языке Promela описаны адресная и широковещательная запись регистра, проверка получателя, обработка неверной CRC, неподдерживаемой функции и ошибочного адреса регистра. Для модели сформулированы и разобраны 14 LTL-свойств, проверяющих сохранность памяти, корректность ответов и завершение обмена.

По результатам законченных поисков все свойства подтверждены, недопустимых конечных состояний не обнаружено. При этом автоматический запуск не смог закончить проверку свойства прогресса за установленное время: его пришлось выполнить отдельно. Повторный поиск завершился без нарушений за 214 секунд. Модель и формулы при этом не изменялись.

Существенную роль сыграли наблюдательные данные и согласование границ транзакции. Снимки регистров, флаги операций, счетчик ответов и уведомления обработчиков позволяют проверять именно результат рассматриваемого запроса. Таким образом, при формализации системы необходимо учитывать не только ее полезные действия, но и то, в какой момент сформулированное свойство должно оцениваться.

Рассмотренная модель описывает только небольшой фрагмент поведения Modbus. Даже при трех Slave, двух регистрах и ограниченном наборе значений большинство проверок сохраняло более 14 миллионов состояний, а проверка прогресса — более 28 миллионов. Успешные поиски суммарно заняли около 19 минут и потребовали до 2 359 Мбайт памяти по отчету SPIN. При расширении состава устройств, диапазонов данных и правил обмена число сочетаний состояний и допустимых взаимодействий будет возрастать. Поэтому верификация более полной системы потребует дальнейшего выбора абстракций и, возможно, раздельной проверки ее частей. Полученный результат показывает корректность выбранного фрагмента в пределах явно заданных предположений.

Список использованных источников

[1] DafterT. Modbus Promela: репозиторий исходного кода [Электронный ресурс]. — GitHub. — URL: https://github.com/DafterT/modbus-promela (дата обращения: 28.10.2026).

[2] Modbus Organization. MODBUS Application Protocol Specification V1.1b3 [Электронный ресурс]. — URL: https://www.modbus.org/file/secure/modbusprotocolspecification.pdf (дата обращения: 28.10.2026).

[3] SPIN. Promela Reference: atomic [Электронный ресурс]. — URL: https://spinroot.com/spin/Man/atomic.html (дата обращения: 28.10.2026).

[4] SPIN. Promela Reference: LTL [Электронный ресурс]. — URL: https://spinroot.com/spin/Man/ltl.html (дата обращения: 28.10.2026).

[5] SPIN. Pan: The Verification Engine [Электронный ресурс]. — URL: https://spinroot.com/spin/Man/Pan.html (дата обращения: 28.10.2026).